

Advanced Topics in Computer Graphics I

Filtering



Prof. Dr. Reinhard Klein

Institute of Computer Science II - Computer Graphics
University of Bonn

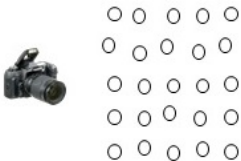
June 18, 2026



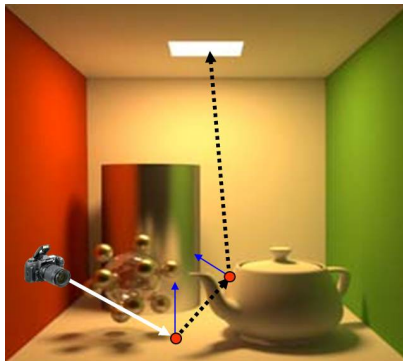


Monte-Carlo-Integration of the Rendering Equation was realized by the Path Tracing algorithm.

For each pixel, we shoot k primary rays.



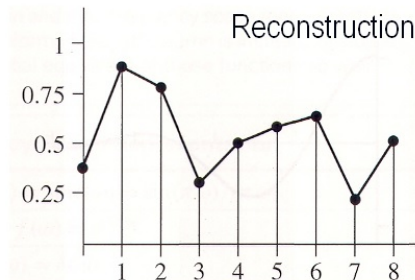
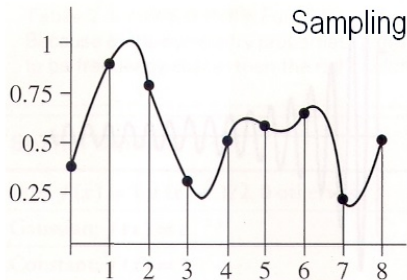
- ▶ $k = 1$ Directly use the computed color
- ▶ $k > 1$ For more than one color, we need a reconstruction filter!





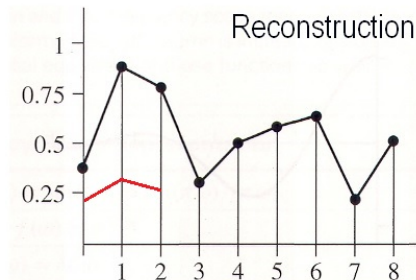
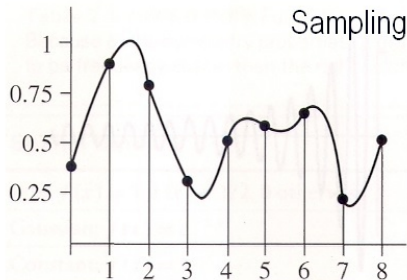
Sampling and Reconstruction





Question: What is meant by sampling and reconstruction?

- ▶ **Sampling:** (Discrete) sampling of an unknown function
- ▶ **Reconstruction:** Reconstruction of the function from samples

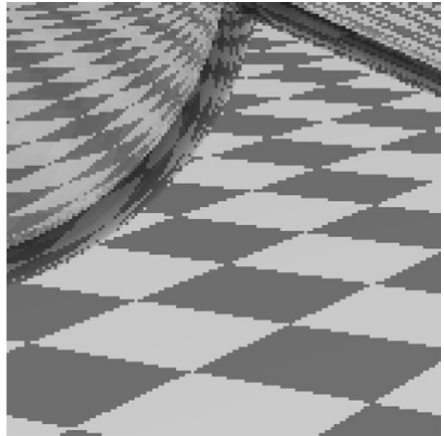
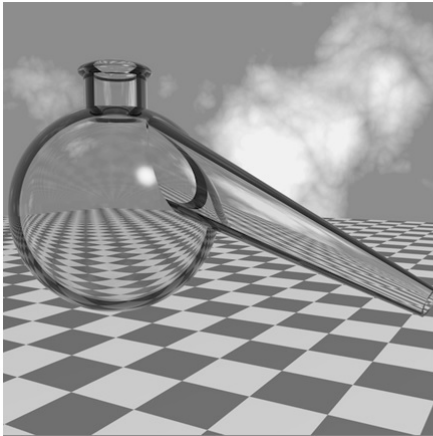


Problem: Reconstruction only approximates the original function

- Reconstruction error
- Aliasing



Example: “Jaggies”

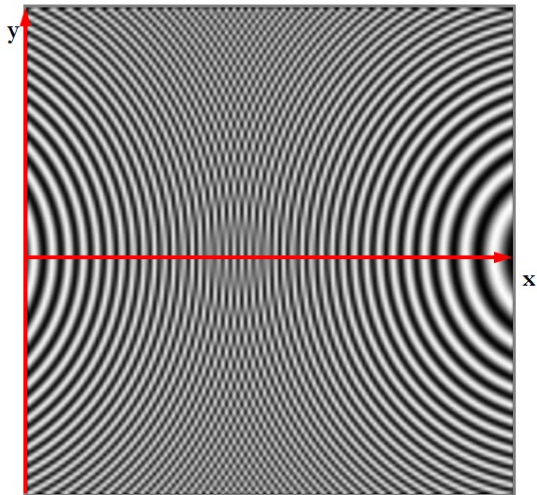




Example: Moiré Pattern

Sampling of the function:

$$\sin(x^2 + y^2)$$





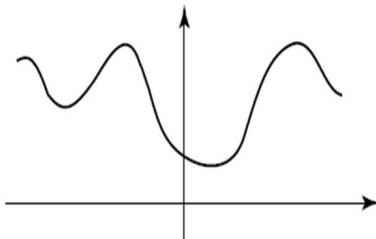
Question: What happens during sampling and reconstruction?

Idea: Fourier Analysis

Given a continuous function (signal) f :

$$f: \mathbb{R} \rightarrow \mathbb{C}.$$

We interpret this function as a point in an infinite-dimensional vector space. The i -th value of f describes the distance of the function to the i -th dimension of this vector space V .





Since our functions are now interpreted as points in a vector space V , we can define a dot product and a norm:

Definition (Dot product between signals)

Let $f, g: \mathbb{R} \rightarrow \mathbb{C}$ be continuous functions. The *dot product* between f and g is defined as

$$\langle f | g \rangle_V := \int_{\mathbb{R}} f(t) \overline{g(t)} dt$$

Definition (Norm of a signal)

Let $f: \mathbb{R} \rightarrow \mathbb{C}$ be a continuous function. The *2-norm* of f is defined as

$$\|f\|_2 := \sqrt{\langle f | f \rangle_V}$$

Furthermore, the following holds:

$$\|f\|_2 = \sqrt{\int_{\mathbb{R}} |f(t)|^2 dt}$$



To analyze a function, we need one important property:

Definition (Space of signals with finite energy)

The set

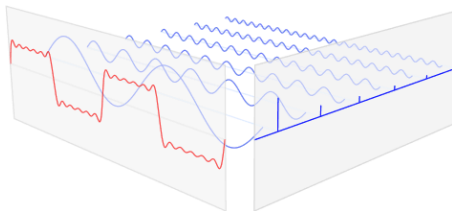
$$L^2(\mathbb{R}) := \{f: \mathbb{R} \rightarrow \mathbb{C} \mid \|f\|_2 < \infty\}$$

is called the *Space of signals with finite energy*.

Typically, all functions which we will consider lie in this space. So, we do not restrict ourselves with this property.

Crucial observation:

Any signal can be decomposed into
an infinite sum of sine and cosine
functions!





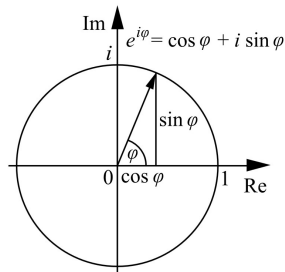
Using this observation, we derive the Fourier Transform:

Lemma (Euler's formula)

The equation

$$e^{ix} = \cos(x) + i \sin(x)$$

is called Euler's formula.



So we can write a complex combination of sine and cosine as an exponential of a complex number. Furthermore, on bounded or periodic domains, complex exponentials

$$e_{\omega}: t \mapsto e^{2\pi i \omega t}$$

form an orthonormal basis. On $\mathbb{R}$, the exponentials are not square-integrable and are therefore not ordinary basis functions of $L^2(\mathbb{R})$; instead, they are interpreted as generalized frequency modes. The Fourier transform measures how strongly f correlates with each frequency ω .



Since this basis is orthonormal, we can compute the component of f to each basis function using the dot product.

Definition (Fourier Transform)

Let $f \in L^2(\mathbb{R})$. The function $\hat{f}: \mathbb{R} \rightarrow \mathbb{C}$ with

$$\hat{f}(\omega) = \int_{\mathbb{R}} f(t) e^{-2\pi i \omega t} dt$$

is called *Fourier Transform* of f .

On the other hand, we can write the signal in this basis. This is called Inverse Fourier Transform:

Definition (Inverse Fourier Transform)

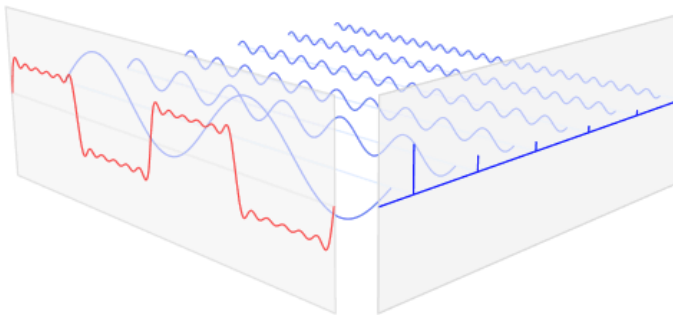
Let $\hat{f} \in L^2(\mathbb{R})$. The mapping $f: \mathbb{R} \rightarrow \mathbb{C}$ with

$$f(t) = \int_{\mathbb{R}} \hat{f}(\omega) e^{2\pi i \omega t} d\omega$$

is called *Inverse Fourier Transform* of $\hat{f}$.



So, we discovered the fundamental formulae to convert a signal between spatial and frequency domain:



Spatial domain

$$f(t) = \int_{\mathbb{R}} \hat{f}(\omega) e^{2\pi i \omega t} d\omega$$

Frequency domain

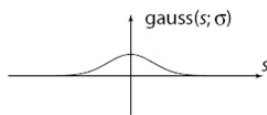
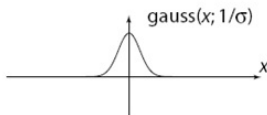
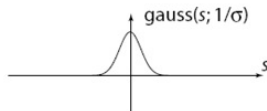
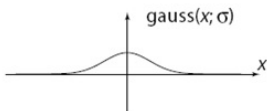
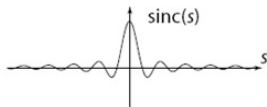
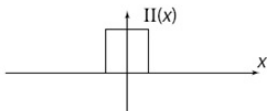
$$\hat{f}(\omega) = \int_{\mathbb{R}} f(t) e^{-2\pi i \omega t} dt$$



Example:

spatial domain

frequency domain





Definition (2D Fourier Transform)

Let $f \in L^2(\mathbb{R}^2)$. The function $\widehat{f}: \mathbb{R}^2 \rightarrow \mathbb{C}$ with

$$\widehat{f}(u, v) = \int_{\mathbb{R}} \int_{\mathbb{R}} f(x, y) e^{-2\pi i(ux+vy)} dx dy$$

is called *Fourier Transform* of f , where u and v are spatial frequencies.
The corresponding mapping

$$f(x, y) = \int_{\mathbb{R}} \int_{\mathbb{R}} \widehat{f}(u, v) e^{2\pi i(ux+vy)} du dv$$

is called *Inverse Fourier Transform* of $\widehat{f}$.

Remarks

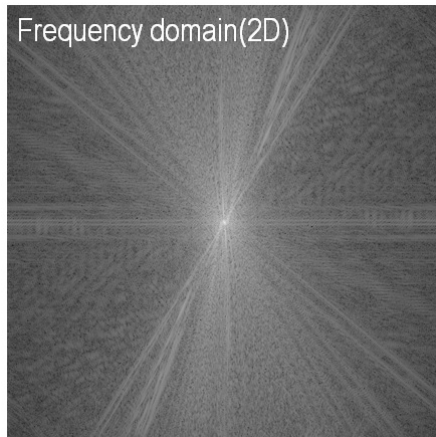
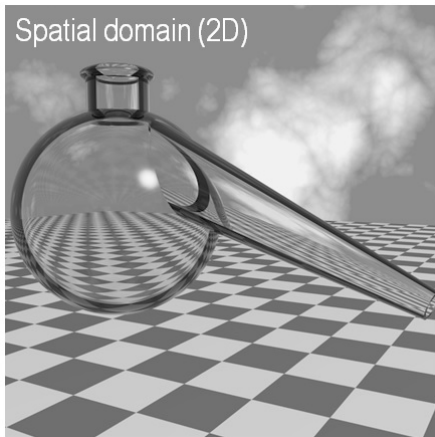
- $\widehat{f}(u, v)$ is complex in general,

$$\widehat{f}(u, v) = \widehat{f}_{\text{Re}}(u, v) + i\widehat{f}_{\text{Im}}(u, v)$$

- $|\widehat{f}(u, v)|$ is called the magnitude spectrum
- $\arctan\left(\widehat{f}_{\text{Im}}(u, v)/\widehat{f}_{\text{Re}}(u, v)\right)$ is called the phase angle spectrum



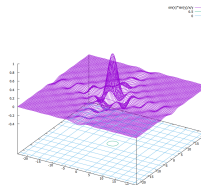
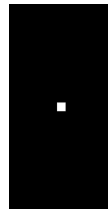
Example:





Example 1: rectangle of height 1 centered at origin with sides of length X and Y

$$\begin{aligned}\hat{f}(u, v) &= \int_{\mathbb{R}} \int_{\mathbb{R}} f(x, y) e^{-2\pi i(ux+vy)} dx dy \\&= 1 \cdot \int_{-\frac{X}{2}}^{\frac{X}{2}} e^{-2\pi i u x} dx \cdot \int_{-\frac{Y}{2}}^{\frac{Y}{2}} e^{-2\pi i v y} dy \\&= \left[\frac{e^{-2\pi i u x}}{-i2\pi u} \right]_{-\frac{X}{2}}^{\frac{X}{2}} \cdot \left[\frac{e^{-2\pi i v y}}{-i2\pi v} \right]_{-\frac{Y}{2}}^{\frac{Y}{2}} \\&= \frac{1}{-2\pi i u} [e^{-i\pi u X} - e^{i\pi u X}] \cdot \frac{1}{-2\pi i v} [e^{-i\pi v Y} - e^{i\pi v Y}] \\&= XY \left[\frac{\sin(\pi X u)}{\pi X u} \right] \left[\frac{\sin(\pi Y v)}{\pi Y v} \right] \\&= XY \cdot \text{sinc}(\pi X u) \cdot \text{sinc}(\pi Y v).\end{aligned}$$





In order to understand filtering, we need one more operator:

Definition (Convolution)

Let $f, g \in L^2(\mathbb{R})$ be two signals. The *convolution* between f and g is defined as

$$(f * g)(t) := \int_{\mathbb{R}} f(t - x) g(x) dx$$

The convolution of two signals is again a signal.

The convolution has the following properties:

Theorem (Algebraic properties of the convolution)

Let $f, g, h \in L^2(\mathbb{R})$ be three signals and $\alpha, \beta \in \mathbb{C}$ be complex numbers. The convolution is associative, commutative and linear:

- ▶ $(f * g) * h = f * (g * h)$
- ▶ $f * g = g * f$
- ▶ $f * (\alpha g + \beta h) = \alpha (f * g) + \beta (f * h)$

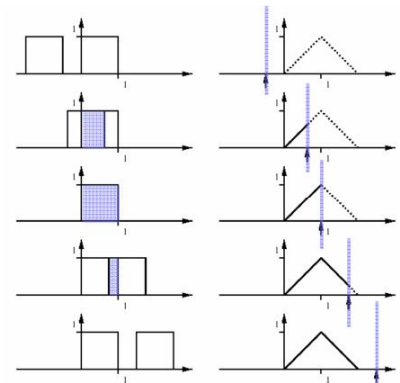


Convolution



Intuitively, the convolution between two signals f and g is the intersected area between the **mirrored and shifted signal f** and the **signal g** .

$$(f * g)(t) := \int_{\mathbb{R}} f(t - x) g(x) dx$$





Convolution

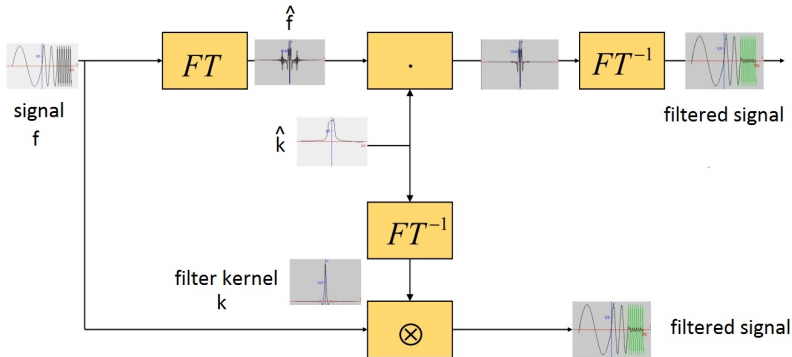


Now, we can formulate the Convolution Theorem:

Theorem (Convolution Theorem)

Let $f, g \in L^2(\mathbb{R})$ be two signals. Then the following holds:

$$\widehat{f * g} = \widehat{f} \cdot \widehat{g}$$



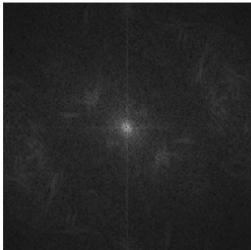
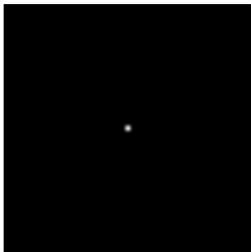
Convolution in spatial domain is equal to point wise multiplication in frequency domain.



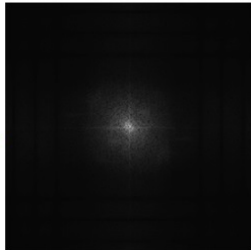
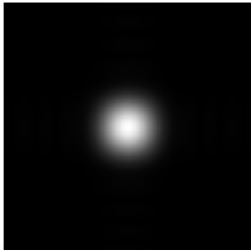
Example:



*



.





Sampling Theorem

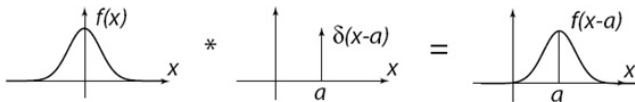


Now, we will derive the Sampling Theorem which will give us the reason why aliasing occurs during sampling.

Consider the Dirac Delta function. If we convolve a function f with δ , we get the function value of f :

$$\begin{aligned}(f * \delta)(x_0) &= \int_{\mathbb{R}} f(x_0 - x) \delta(x) dx \\ &= \int_{\mathbb{R}} \delta(x_0 - x) f(x) dx \\ &= f(x_0)\end{aligned}$$

Similarly, if we convolve a shifted Dirac Delta function with f , we get a shifted f :





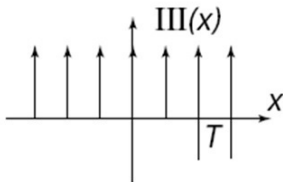
Sampling Theorem



Now, we consider the Shah-function $\text{III}(x)$ which is a sum of shifted Delta functions:

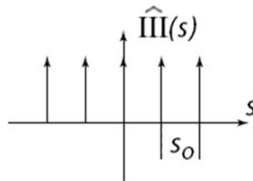
Spatial domain

$$\text{III}(x) = \sum_{n=-\infty}^{\infty} \delta(x - nT)$$



Frequency domain

$$\hat{\text{III}}(s) = \sum_{n=-\infty}^{\infty} \delta(s - ns_0) \quad s_0 = 1/T$$



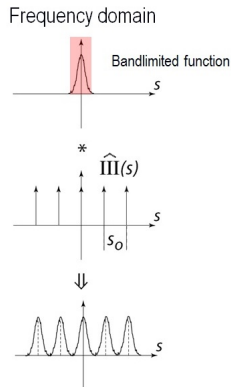
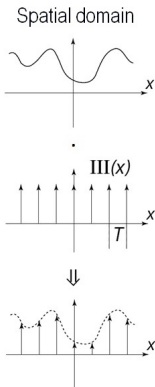
Observation: The Shah-function transformed to the frequency domain is again a Shah-function!



Sampling Theorem



Sampling can be regarded as point-wise multiplication of the Shah-function to the continuous signal f :



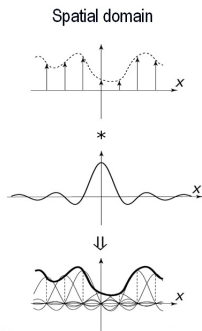
Observation: In frequency domain, this leads to copies of $\hat{f}$!



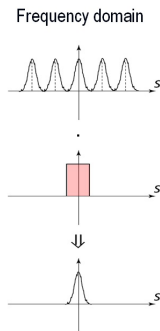
Sampling Theorem



To reconstruct the original signal, we need to point-wise multiply the signal in frequency domain with a box filter. In spatial domain, this is a convolution with the sinc-function:



$$\begin{aligned}\tilde{f} &= (f(x) \cdot \text{III}(x)) * \text{sinc}(x) \\ &= \sum_{n=-\infty}^{\infty} \text{sinc}(x - n) f(n)\end{aligned}$$



$$\tilde{f} = (\hat{f}(s) * \hat{\text{III}}(s)) \cdot \Pi(s)$$



Sampling Theorem

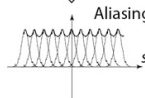
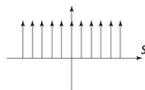
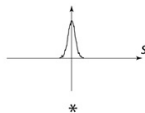
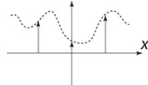
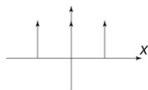
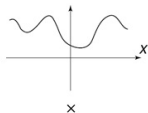


Aliasing occurs if the sampling of the function was not dense enough. Then the copies of $\hat{f}$ in the frequency domain **overlap** and cause artifacts during reconstruction:

Large sample distance
in spatial domain



Small distance in
frequency domain



Aliasing

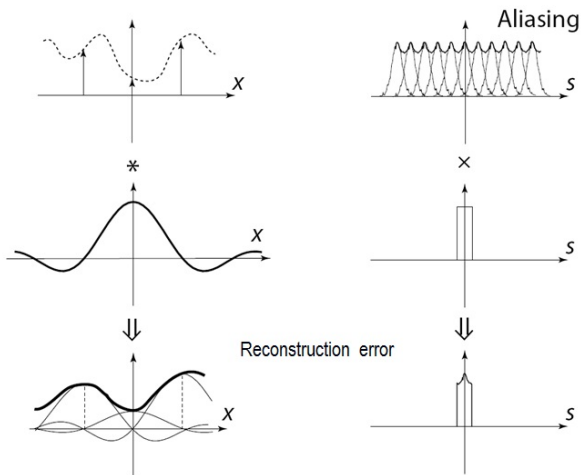
High frequency details overlap
with low frequency ones



Sampling Theorem



Aliasing occurs if the sampling of the function was not dense enough. Then the copies of $\hat{f}$ in frequency domain **overlap** and cause artifacts during reconstruction:





Theorem (Sampling Theorem¹)

If the function $f \in L^2(\mathbb{R})$ is band-limited, i.e.

$$\exists \omega_{\max} : \forall \omega > \omega_{\max} : \hat{f}(\omega) = 0,$$

then the function f can be reconstructed from without any error from its uniform samples $f(m\Delta t)$, $m \in \mathbb{Z}$ provided that the sampling frequency $f_s = \frac{1}{\Delta t}$ satisfies

$$f_s \geq 2 \cdot \omega_{\max}.$$

The value $2 \cdot \omega_{\max}$ is the Nyquist rate. $\frac{f_s}{2}$ is called Nyquist Frequency. It is the highest frequency that can be unambiguously reconstructed from uniformly sampled data, assuming ideal low-pass reconstruction and that the original signal is band-limited.

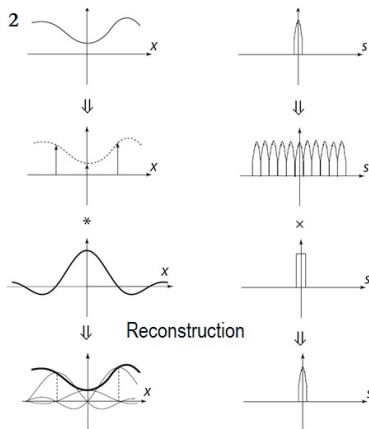
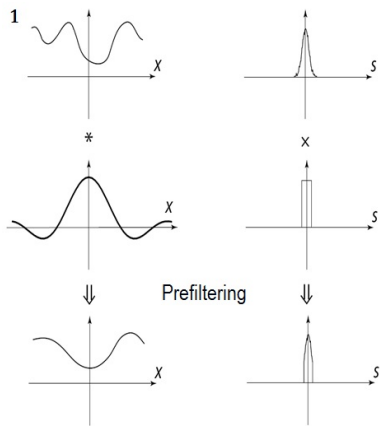
Anti-Aliasing-Techniques: To avoid aliasing, there are two major techniques:

- ▶ **Prefiltering:** Band-limit the signal via filtering
- ▶ **Supersampling:** Increase the sampling rate

¹C. E. Shannon. "Communication in the presence of noise". In: *Proceedings of the IRE* 37.1 (1949), pp. 10–21.



Technique 1: Prefiltering



"blurring is better than aliasing"

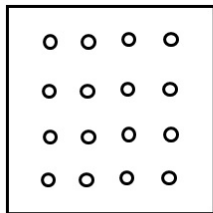


- ▶ **Natural Perception:** Blurring mimics real-world defocusing, making it less jarring and more interpretable to the human eye compared to the artificial patterns of aliasing.
- ▶ **Information Preservation:** Blurring maintains underlying content in a smoothed form, while aliasing introduces false information, which can be misleading especially in critical applications like medical imaging.
- ▶ **Visual Appeal:** Blurring is subtler and often more aesthetically pleasing, whereas aliasing creates harsh and distracting visual artifacts.
- ▶ **Post-Processing Potential:** Blur is a controlled low-pass error; aliasing folds high frequencies into false low frequencies, which is often harder to interpret or remove.
- ▶ **Predictable Error:** Blurring produces consistent and uniform errors, in contrast to the unpredictable and variable high-frequency artifacts caused by aliasing.



Technique 2: Supersampling

- ▶ **Assumption:** Energy (Fourier Transform) of a function decreases with increasing frequency
- ▶ **Supersampling:** Increase of sampling rate $\rightarrow$ Reduction of “aliasing energy”
- ▶ **Resampling:** Low-pass filter to obtain desired discretization

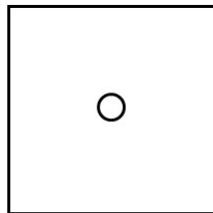


Samples

$$Pixel = \sum_s w_s \cdot Sample_s$$



Resampling



Pixel



Example:

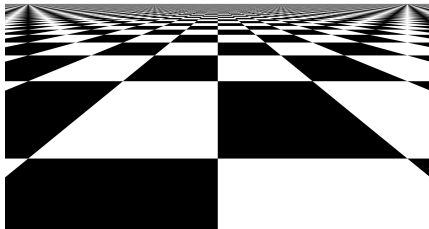
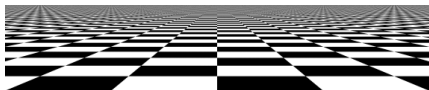


Figure: Exact (analytic)

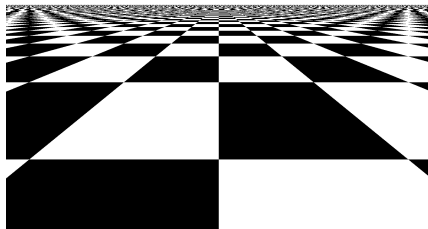


Figure: Point sampling



Example:

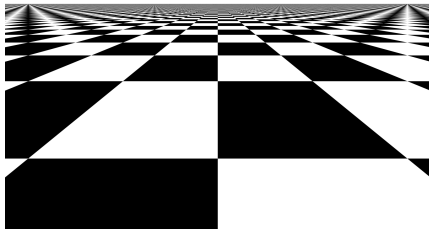
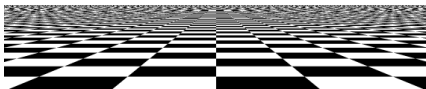
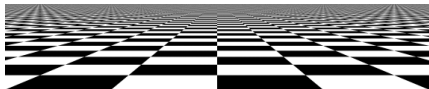


Figure: Exact (analytic)

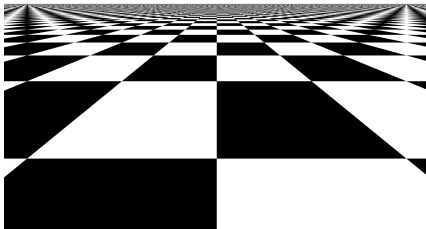


Figure: 4x4 Regular super sampling



Example:

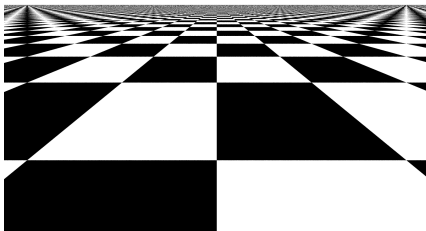
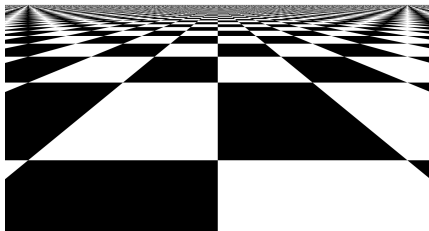
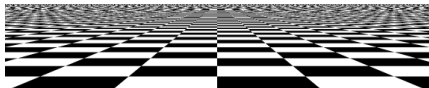


Figure: 4x4 Regular super sampling

Figure: 4x4 Jittered super sampling

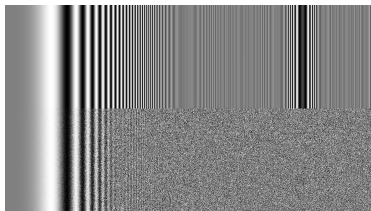


► Uniform sampling

- The spectrum of uniformly spaced samples is also a set of uniformly spaced spikes
- Multiplying the signal by the sampling pattern corresponds to placing a copy of the spectrum at each spike (in freq.space) resulting in coherent (structured) aliasing
- Structures are noticeable!

► Non-uniform sampling

- Samples at non-uniform locations have a different spectrum; a single spike plus noise
- Sampling a signal in this way converts aliases into broadband noise
- Noise is incoherent (structureless) and much less objectionable



- The impulse train is modified as

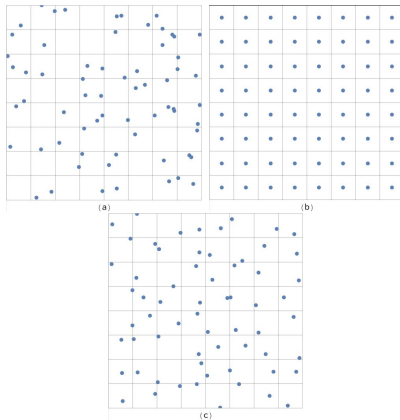
$$\sum_{i=-\infty}^{\infty} \delta\left(x - \left(i + \frac{1}{2} - \xi\right)T\right),$$

where $\xi \in [0, 1]$ is a random number.

- It turns regular aliasing artifacts into noise, which is less distracting to the human visual system.



- ▶ The **random pattern** is an ineffective pattern, with many clumps of samples that leave large sections of the image poorly sampled.
- ▶ A **uniform stratified pattern** is better distributed but can exacerbate aliasing artifacts
- ▶ A **stratified jittered pattern** turns aliasing from the uniform pattern into high-frequency noise while still maintaining the benefits of stratification



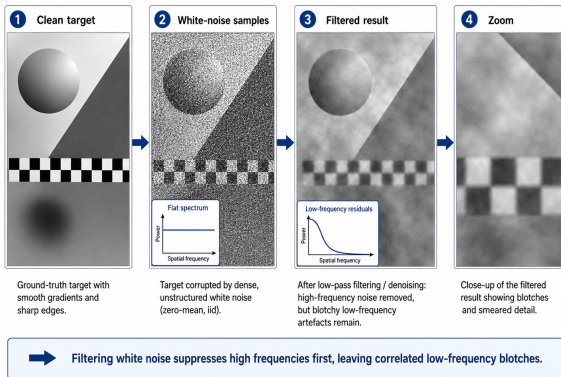


Recall from our basic MC lecture how unstratified random sampling projects errors across the frequency domain. The geometric distribution of these samples fundamentally dictates filtering performance.

White Noise (Random/Clumpy)

- ▶ Noise energy is distributed evenly across all frequencies.
- ▶ **The Filtering Problem:** Creates low-frequency structural clusters.
- ▶ Image-space denoisers cannot differentiate low-frequency noise clumps from true soft lighting features, leading to blotchy artifacts.

White noise + filtering -> low-frequency artefacts

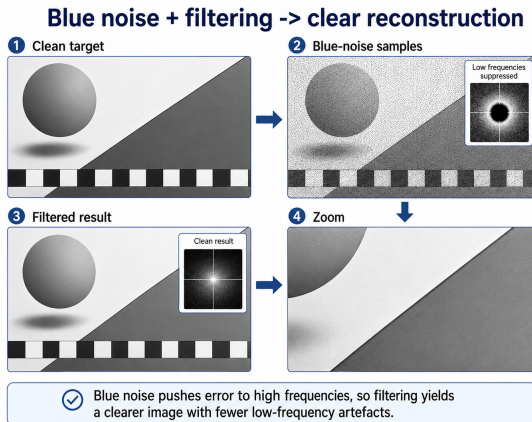




By shifting to the low-discrepancy and stratified patterns we analyzed previously, we completely alter the mathematical landscape for our reconstruction filter.

Blue Noise & Low-Discrepancy

- ▶ Projections are maximally separated; sample variance is pushed entirely into **high frequencies**.
- ▶ **The Filtering Advantage:** High-frequency error manifests as fine per-pixel grain.
- ▶ Local spatial kernels (Cross-Bilateral, A-Trous, Neural KPN) remove this fine grain effortlessly.





Modern Denoising



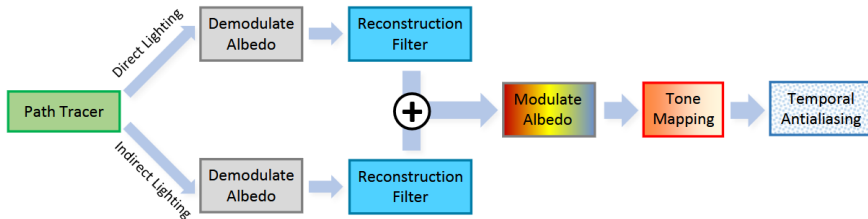
We have seen that **non-uniform sampling** converts structured aliasing into high-frequency noise that is less perceptually objectionable.

Modern real-time path tracers push this idea to the extreme:

- ▶ Render at **1–2 samples per pixel (spp)** per frame. Each pixel receives at most one or two stochastic light paths.
- ▶ The resulting image contains **severe Monte Carlo noise**, but this noise is incoherent and can be mathematically filtered.
- ▶ A **denoiser** reconstructs a plausible clean image from the noisy samples, using additional geometric information stored in secondary targets.



Key insight: The rendering budget freed from supersampling can be reinvested into *ray quality* (complex BSDFs, longer paths, area light sampling) while the denoiser handles



Inputs per frame:

- ▶ Noisy radiance buffer (1–2 spp)
- ▶ G-Buffer channels (albedo, normals, depth, motion)
- ▶ Previous denoised frame history

Pipeline stages:

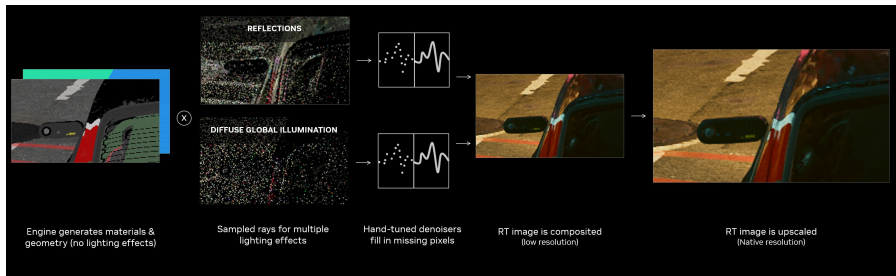
1. G-Buffer auxiliary generation
2. Temporal reprojection & accumulation
3. Variance estimation
4. Edge-preserving spatial filtering



The Traditional Real-Time Ray Tracing Pipeline



Classical real-time hybrid pipeline: geometry and materials are produced by the rasteriser; each lighting effect (reflections, diffuse GI, ...) is sampled separately by a dedicated ray-tracing pass and then cleaned up by its own hand-tuned denoiser.



Separate denoisers per lighting effect means separate parameter tuning, separate failure modes, and separate runtime budgets.

Compositing happens at low internal resolution; the composited image is then handed to the upscaler.

Upscaling expects a clean, detail-rich input — which the prior denoising step may not guarantee.



The Challenge: A surface texture (Albedo, ρ) is crisp and noise-free, but the stochastically sampled incoming light (Illumination, L) is highly noisy.

- ▶ Blending raw pixel colors directly ($C = \rho \cdot L$) smooths out vital texture details (wood grain, text) along with the noise.

The Solution: Strip the texture out before filtering, and re-apply it afterwards.

1. **Demodulation (Pre-Division):** Divide the noisy pixel color by the noise-free G-Buffer albedo to isolate the raw illumination field:

$$L_{\text{noisy}} = \frac{C_{\text{noisy}}}{\rho + \varepsilon}$$

2. **Denoising:** Run spatial/temporal filters purely on the flat L_{noisy} illumination buffer. Texture details are completely safe from blurring.
3. **Re-modulation:** Multiply the filtered, clean illumination ($\tilde{L}$) back by the sharp texture albedo to produce the final pixel color:

$$\tilde{C} = \rho \cdot \tilde{L}$$



Raw radiance values mix light paths with fundamentally different physical properties and mathematical noise frequencies:

Direct Lighting Pass

- ▶ Light traveling directly from a emitter to the visible surface point.
- ▶ **Noise Profile:** Sharp shadow boundaries (penumbras) and high-frequency specular highlights.
- ▶ **Filtering Strategy:** Requires a **narrow spatial footprint** with strict edge-stopping criteria to prevent shadows or bright highlights from bleeding across geometry.

Indirect Lighting Pass

- ▶ Global illumination (GI), ambient bounces, and diffuse color bleeding.
- ▶ **Noise Profile:** Low-frequency, low-energy, highly blotchy low-spp noise.
- ▶ **Filtering Strategy:** Naturally smooth with few sharp outlines; can safely use a **wide spatial footprint** (à-trous with wide strides) to clear blotches quickly.

Execution Rule: Direct and indirect buffers are demodulated independently, filtered using customized kernels, and re-modulated before a final summation pass.



Motivation: A pixel's noisy radiance value alone carries no information about whether its neighbours belong to the *same surface*. Blending across geometry boundaries produces characteristic *halo* or *ghosting* artifacts.

Definition (Geometry Buffer (G-Buffer))

A *G-Buffer* is a collection of screen-space raster targets, each storing a geometric or material attribute of the closest visible surface point for every pixel. Produced as a by-product of the rasterization or primary-ray pass, G-Buffer reads impose **no additional ray cost**.

G-Buffer data serves two orthogonal roles in the denoising pipeline:

- ▶ **Edge-stopping signal:** prevents blending across depth discontinuities and normal discontinuities.
- ▶ **Temporal correspondence:** motion vectors enable accurate reprojection of samples from frame $t - 1$ to frame t .



G-Buffer Components

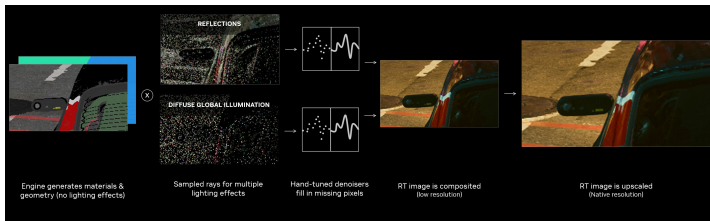


Figure: NVIDIA: G-buffer channels generated alongside light effect samples.

- ▶ **Albedo / Base Color:** Diffuse reflectance $\rho \in [0, 1]^3$, decoupled from illumination. Allows filtering irradiance independently before color re-multiplication.
- ▶ **World-Space Normals** $\mathbf{n} \in \mathbb{R}^3$: Encoded as (N_x, N_y, N_z) . Diverging normals indicate a geometric boundary — the primary edge-stop signal.
- ▶ **Linear Depth** $z \in \mathbb{R}$: World-space depth or position. Used for disocclusion detection and depth-based edge stopping.
- ▶ **Screen-Space Motion Vectors** $(\Delta u, \Delta v)$: Per-pixel 2-D displacement to the pixel's location in the previous frame.



Problem: A standard Gaussian filter smooths noise but also blurs structural edges. Its weight depends strictly on spatial distance:

$$w_d(\mathbf{p}, \mathbf{q}) = \exp\left(-\frac{\|\mathbf{p} - \mathbf{q}\|^2}{2\sigma_d^2}\right).$$

Definition (Bilateral Filter²)

The *bilateral filter* extends the Gaussian kernel with a **range kernel** w_r that penalises large differences in pixel values:

$$\tilde{I}(\mathbf{p}) = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in \mathcal{N}(\mathbf{p})} I(\mathbf{q}) \exp\left(-\frac{\|\mathbf{p} - \mathbf{q}\|^2}{2\sigma_d^2}\right) \exp\left(-\frac{\|I(\mathbf{p}) - I(\mathbf{q})\|^2}{2\sigma_r^2}\right),$$

where $W_{\mathbf{p}}$ is the local normalisation factor.

The range kernel **automatically attenuates** contributions from neighbours across an intensity edge: if $I(\mathbf{p}) \approx I(\mathbf{q})$ (same surface), $w_r \approx 1$; if they differ strongly (edge), $w_r \approx 0$.

²C. Tomasi and R. Manduchi. "Bilateral Filtering for Gray and Color Images". In: *Proceedings of the Sixth International Conference on Computer Vision (ICCV)*. IEEE, 1998, pp. 839–846.



Problem: A standard bilateral filter uses the noisy image I itself as the guide. For Monte Carlo denoising, this is unreliable: random variance can create false intensity boundaries, causing the filter to preserve noise.

Idea: Guide the filter with clean G-Buffer signals instead of noisy color. Normals and depth provide stable information about real geometric edges.

Definition (Cross-Bilateral / Joint Bilateral Filter)

For a pixel p and neighbourhood $\mathcal{N}(p)$:

$$\tilde{I}(p) = \frac{1}{W_p} \sum_{q \in \mathcal{N}(p)} I(q) w_d(p, q) w_n(p, q) w_z(p, q).$$

$$w_n(p, q) = \exp\left(-\frac{\max(0, 1 - \mathbf{n}_p \cdot \mathbf{n}_q)}{\sigma_n^2}\right), \quad w_z(p, q) = \exp\left(-\frac{|z_p - z_q|}{\sigma_z}\right).$$

Interpretation: average nearby samples only if their normals and depths suggest they belong to the same surface.

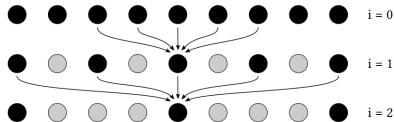
Challenge 1: Computational Cost

A full spatial $r \times r$ evaluation scales quadratically as $\mathcal{O}(r^2)$ per pixel. For a target radius $r = 32$, this demands over 1024 lookups.

Solution: *À-trous* (“with holes”) decomposition — apply a sequence of hierarchical passes with an expanding stride 2^i , using a compact 5×5 footprint. The effective filter radius grows exponentially while keeping operations linear.

Challenge 2: Data Insufficiency

At 1 spp, a single frame's spatial neighbourhood does not contain enough random paths to resolve detailed multi-bounce lighting equations.



Solution: Extend reconstruction into the *spatiotemporal* domain — accumulate independent samples along time tracks via motion vectors.



The Spatiotemporal Problem



Why spatial-only filtering at 1 spp is fundamentally limited:

- ▶ Each pixel has **exactly one** Monte Carlo path sample. No local statistical ensemble exists to track variance directly.
- ▶ A wide spatial kernel smooths noise but systematically destroys fine geometry if edge thresholds are loose.
- ▶ A tight kernel preserves edges but leaves high-frequency noise clusters behind.

Key observation: Across frames, the *same surface point* is observed repeatedly over time. If we trace its screen projection, independent samples can be accumulated temporally.



Goal: Given pixel p at frame t , pinpoint its exact tracking coordinate $\hat{p}$ in frame $t - 1$.

Definition (Temporal Reprojection via Motion Vectors)

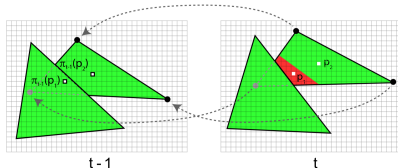
Let $(\Delta u_p, \Delta v_p)$ be the screen displacement vector at pixel p . The reprojected historical position is evaluated as:

$$\hat{p} = p - (\Delta u_p, \Delta v_p).$$

For rigid static geometry, motion is computed directly from projection matrices. For animated objects, a dedicated velocity buffer stores dynamic velocity updates.

Validation check — reprojection is immediately invalidated if structural deviations occur:

- ▶ Depth delta test exceeds boundary ($\rightarrow$ *disocclusion*).
- ▶ Normal divergence test fails ($n_p \cdot n_{\hat{p}} < \tau$).





Following validation, the current frame sample is blended with historical contexts using an **Exponential Moving Average (EMA)** loop:

Definition (Temporal Accumulation)

Let $\tilde{C}_{t-1}(\hat{p})$ be the bilinearly interpolated color history and $C_t(p)$ the newly rendered 1 spp sample. The updated accumulation field $\tilde{C}_t(p)$ corresponds to:

$$\tilde{C}_t(p) = \alpha \tilde{C}_{t-1}(\hat{p}) + (1 - \alpha) C_t(p), \quad \alpha \in [0, 1).$$

The effective sample footprint grows geometrically, approaching an equivalent static convergence rate of $\frac{1}{1-\alpha}$ samples.

Weight parameters:

- ▶ Production standard: $\alpha \approx 0.8\text{--}0.95$ (5 to 20 effective frames).
- ▶ On disocclusion: $\alpha = 0$, dropping history to prevent ghosting.

The core trade-off:

- ▶ $\alpha \rightarrow 1$: Maximum noise reduction, heavy lag/smearing risks.
- ▶ $\alpha \rightarrow 0$: Responsive, but limits sample integration to spatial context.

Tracking accumulated moving color moments over time yields a dynamic, running estimation of localized sample variance per pixel.

Definition (Spatiotemporal Variance Estimation)

Accumulate the first and second moments of luminance concurrently using matched EMA coefficients:

$$\tilde{\mu}_1(\mathbf{p}) = \alpha \tilde{\mu}_{1,t-1}(\hat{\mathbf{p}}) + (1 - \alpha) \ell(\mathbf{p}), \quad \tilde{\mu}_2(\mathbf{p}) = \alpha \tilde{\mu}_{2,t-1}(\hat{\mathbf{p}}) + (1 - \alpha) \ell(\mathbf{p})^2,$$

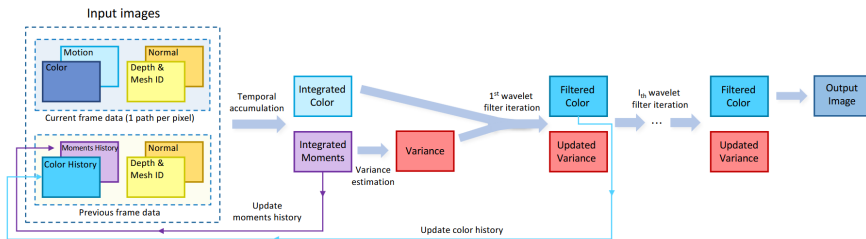
where $\ell(\mathbf{p}) = \text{lum}(C_t(\mathbf{p}))$. The per-pixel variance estimate is evaluated as:

$$\hat{\sigma}^2(\mathbf{p}) = \tilde{\mu}_2(\mathbf{p}) - \tilde{\mu}_1(\mathbf{p})^2.$$

If the history buffer drops due to disocclusion ($\alpha = 0$), a local spatial 7×7 window is evaluated to bootstrap the variance field $\hat{\sigma}^2$.



SVGF unifies temporal reprojection and dynamic local variance tracking to drive an edge-preserving filter without hand-tuned global coefficients.



Stage 1: Reprojection

Warp previous frame coordinates via screen motion vectors and accumulate running luminance moments using causal EMA loops.

Stage 2: Estimation

Evaluate the current per-pixel spatiotemporal variance field ($\hat{\sigma}^2$) directly from the accumulated moments.

Stage 3: À-Trous Filter

Execute 5 hierarchical wavelet filtering passes where the luminance edge-stop denominator is actively scaled by local variance.

³C. Schied et al. "Spatiotemporal Variance-Guided Filtering: Real-Time Reconstruction for Path-Traced Global Illumination". In: *Proceedings of High Performance Graphics (HPG)*. ACM, 2017, pp. 1–12.



Each hierarchical à-trous layer applies an expanding footprint with stride 2^i ($i \in [0, 4]$), scaling spatial support to a wide 32-pixel area.

Definition (Variance-Guided Edge-Stopping Function (SVGF))

At wavelet layer i , the integrated luminance edge-stopping weight corresponds to:

$$w_\ell(\mathbf{p}, \mathbf{q}) = \exp\left(-\frac{|\ell(\tilde{C}^{(i)}(\mathbf{p})) - \ell(\tilde{C}^{(i)}(\mathbf{q}))|^2}{\sigma_\ell^2 \hat{\sigma}^2(\mathbf{p}) + \varepsilon}\right).$$

Physical behavior:

- ▶ In high-variance regions where noise is severe, $\hat{\sigma}^2(\mathbf{p})$ is **large** → The denominator expands, $w_\ell \rightarrow 1$, forcing **aggressive spatial blur**.
- ▶ In clean, stable regions, $\hat{\sigma}^2(\mathbf{p})$ is **small** → The denominator shrinks, tightening boundaries to **preserve spatial features**.

Temporal accumulation introduces two failure modes that must be handled explicitly. Coherence visualisations indicate that disocclusions concentrate at structural edges⁴.



(a) Parthenon



(b) Coherence



(c) Heroine



(d) Coherence



(e) Ninja



(f) Coherence

Disocclusion Handling: A wall revealed behind an object samples incorrect history at $\hat{p}$. **Detection:** Depth or normal vector mismatches. **Response:** Set $\alpha = 0$, dropping history. Rely heavily on the wide spatial pass for this frame.

Ghosting Prevention: Moving items leave trails if dynamic historical light contributions leak forward.

Detection: Color-space Axis-Aligned Bounding Box (AABB) clamping.

Response: Adaptively reduce α if current values violate historic bounds.

⁴D. Nehab et al. "Accelerating Real-Time Shading with Reverse Reprojection Caching". In: *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Graphics Hardware*. 2007, pp. 25–35.



The exponential moving average (EMA) feedback loop used in SVGF is not just an intuitive engineering heuristic—it is a streaming screen-space implementation of **Importance Resampling**.

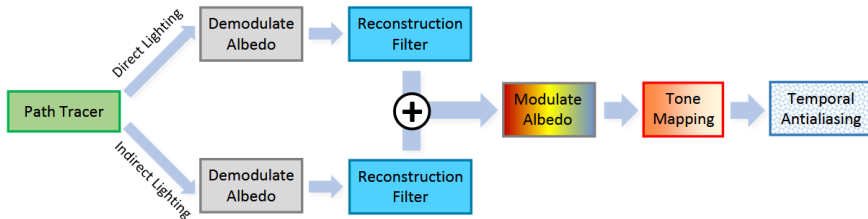
The Continuous Sample Pool

- ▶ Recall from our basic MC lecture that **Sampling Importance Resampling (SIR)** maintains a pool of candidates and selects them based on normalized weights ($\omega = \pi/g$).
- ▶ In SVGF, the accumulated historical color buffer acts as a **temporal sample pool** tracking the underlying light transport field over time.

Disocclusion as Weight Collapse

When a camera movement reveals a hidden wall:

- ▶ The local target distribution π shifts instantly.
- ▶ The importance weights of all historical samples at that tracking pixel **collapse to zero**.
- ▶ Setting $\alpha = 0$ is mathematically a **pruning step** that discards the invalid historical pool and forces an immediate re-sampling from the current frame's proposal.



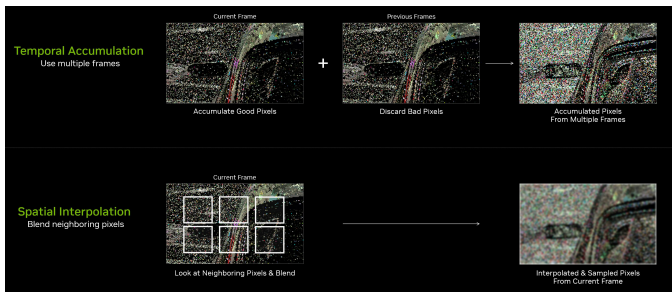
- ▶ **High Efficiency:** Runs entirely on the GPU via compute shaders; total reconstruction overhead < 2 ms at 1080p.
- ▶ **Zero Extra Ray Cost:** Requires only standard, commodity G-Buffer targets generated during primary visibility hits.
- ▶ **Dynamic Robustness:** Handles complex moving characters and camera transforms smoothly via per-pixel motion field offsets.
- ▶ **Temporal Convergence:** Achieves high frame-to-frame sample stability, eliminating objectionable "boiling" artifacts.



Hand-Tuned Denoisers: Two Core Strategies



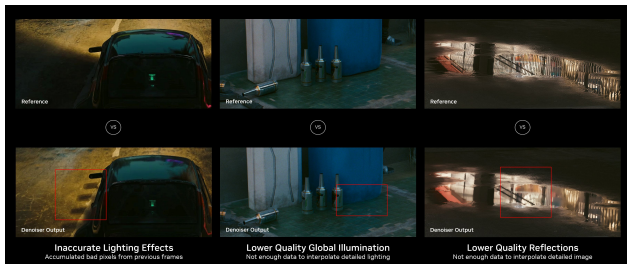
Every hand-crafted real-time denoiser combines exactly two complementary mechanisms to reconstruct a plausible pixel value from sparse ray samples:



Temporal accumulation (top row): reproject and α -blend history from previous frames — the mechanism formalised as the SVGF EMA loop.

Spatial interpolation (bottom row): blend neighbouring pixels within the current frame — the mechanism formalised as the cross-bilateral / à-trous kernel filter.

Fixed, globally-tuned parameters fail on content that lies outside the hand-designed operating envelope:



Inaccurate lighting: bad pixels from previous frames survive temporal blending when the disocclusion test is too lenient.

Lower-quality GI: the spatial filter kernel is too narrow to reconstruct detailed indirect lighting structures from low-spp input samples.

Lower-quality reflections: specular detail is lost because the hand-tuned edge thresholds are calibrated for diffuse noise profiles.

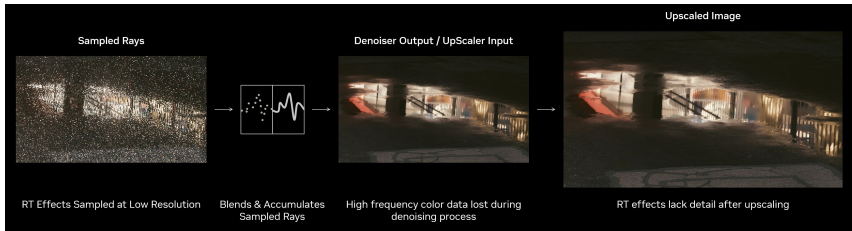
The Structural Bottleneck: A single set of global, hand-crafted parameter bounds ($\sigma_\ell, \sigma_n, \sigma_z, \alpha$) cannot simultaneously adapt to diffuse regions, sharp specular reflections, and sudden visibility boundaries.



Why Denoising and Upscaling Conflict



There is a fundamental tension between the denoising and upscaling steps in the traditional real-time graphics pipeline:



Denoising removes high-frequency energy by design — it acts as a low-pass filter operation. The noisy but information-rich 1 spp input is replaced by a smooth, blurred estimate.

Upscaling requires high-frequency detail to accurately reconstruct a plausible high-resolution target image. If that micro-detail was already discarded, the upscaler outputs soft, flat surfaces.

Why transition to machine learning solutions?

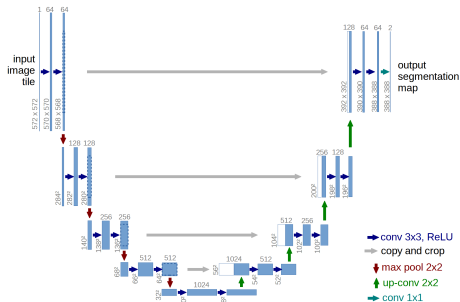
- ▶ Heuristic thresholds are globally fixed; they cannot adapt to localized materials or noise profiles.
- ▶ Local kernels lack global context, causing them to fail near complex illumination structures.
- ▶ Machine learning networks can replace hand-crafted heuristics by learning non-linear, data-driven edge-stopping functions.

The problem is formalized as a supervised multi-channel image translation task:

$$\underbrace{I_{\text{noisy}}}_{1-2 \text{ spp}} + \underbrace{G}_{\text{Albedo, Normals, Depth}} \xrightarrow{\mathcal{F}_{\Theta}} \underbrace{\tilde{I}}_{\text{Denoised Target}}$$

U-Net Architecture for Image Denoising⁵

The dominant backbone in production denoisers is the **U-Net**, an encoder-decoder with lateral skip connections to capture multi-scale context:



- **Encoder:** Successive convolutions and pooling operations downsample the feature field, expanding latent channels to capture wide contextual structures.
- **Decoder:** Progressive upsampling layers restore high-resolution details.
- **Skip Connections:** Route early high-frequency spatial features directly across the bottleneck to prevent structural blurring.

⁵O. Ronneberger et al. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, 2015, pp. 234–241.



Input channels:

- ▶ Noisy radiance $\hat{L}$ (3 ch.)
- ▶ Albedo ρ (3 ch.)
- ▶ World-space normals n (3 ch.)
- ▶ Linear depth z (1 ch.)

Loss function: combination of

- ▶ $\mathcal{L}_1$ or $\mathcal{L}_2$ in **tonemapped** (HDR-aware) space.
- ▶ Perceptual loss (SSIM, LPIPS) to penalise blur without sharp-edge hallucination.

Training data:

- ▶ Pairs of (1 spp, $N \cdot 1024$ spp) renders from diverse synthetic scenes.
- ▶ HDR reference images from offline path tracing.
- ▶ Data augmentation: random crops, exposure changes, scene parameter randomisation.

Inference: once trained, the network runs at **fixed cost** regardless of scene complexity, typically 1–3 ms on a modern GPU via TensorRT execution wrappers.



How Does a Neural Denoiser Compute a Pixel? Two Strategies

A deep network processing inputs can map structural properties using two fundamentally different output paradigms:

Strategy 1: Direct Prediction

$$\tilde{I}(\mathbf{p}) = \mathcal{F}_{\theta}(I(\mathbf{p}), G(\mathbf{p}))$$

The network maps pixel features *directly* to an output color value vector.

Problems:

- ▶ Output is *unconstrained* — the network can predict unmeasured colors.
- ▶ Risk of out-of-range color hallucinations, negative values, or severe clipping.
- ▶ Tends to produce **blurry results** via spatial statistical mean convergence.

Strategy 2: Kernel Prediction

$$\tilde{I}(\mathbf{p}) = \sum_{\mathbf{q} \in \mathcal{N}(\mathbf{p})} K_{\mathbf{p}}(\mathbf{q}) \cdot I(\mathbf{q})$$

The network predicts a *set of blending weights* over a local neighborhood; the renderer evaluates the final color as a weighted sum.

Advantages:

- ▶ Enforces a strict **convex-hull constraint** via normalized non-negative weights ($\sum K = 1, K \geq 0$).
- ▶ Mathematical **positivity and boundedness** are preserved by construction.
- ▶ Produces **sharper**, structurally accurate filters aligned with the noisy input samples.



SVGF replaced per-frame noise with a hand-designed feedback loop. The logical next step: **replace the hand-crafted loop with a learned one.**

SVGF (explicit equations)

- ▶ $\tilde{C}_t = \alpha \tilde{C}_{t-1} + (1 - \alpha) C_t$
- ▶ α fixed; disocclusion by depth/normal test
- ▶ Variance estimated from moment EMA
- ▶ Kernel width from hand-tuned $\sigma_\ell, \sigma_n, \sigma_z$

Recurrent neural network (learned)

- ▶ Hidden state h_t replaces accumulated colour
- ▶ Temporal blending learned end-to-end from data
- ▶ Disocclusion handled implicitly via gated memory
- ▶ Target color metrics optimized from h_t ; no manual σ parameters

Key question: how do we feed temporal history into the network efficiently?

Answer: Recurrent connections at the *bottleneck* of the U-Net using **ConvLSTM** or **GRU** cells — the approach introduced by Chaitanya et al. (2017).



Chaitanya et al. 2017 introduced a recurrent neural architecture to enforce spatiotemporal stability in interactive path-tracing loops⁶:

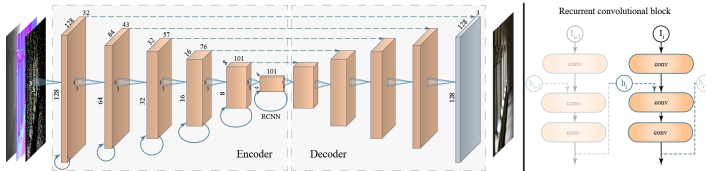


Figure: Recurrent state cell feedback loops at the latent bottleneck layer.

Core architecture loop:

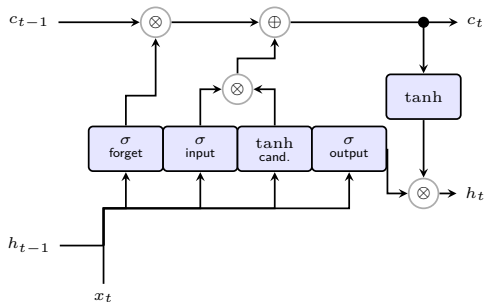
- ▶ A convolutional autoencoder compresses single-frame buffers into low-dimensional latent vectors z_t .
- ▶ A **ConvLSTM cell** at the structural bottleneck captures temporal history:

$$h_t = \text{ConvLSTM}(z_t, h_{t-1}).$$

- ▶ Decoder reconstructs output fields directly from hidden state buffers.

⁶C. R. A. Chaitanya et al. "Interactive Reconstruction of Monte Carlo Image Sequences using a Recurrent Denoising Autoencoder". In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 36.4 (2017), 98:1–98:12.

Standard LSTM Cell Architecture



Input (x_t):

Cell State (c_t):

Runs straight down the entire chain with minor linear updates.;

Hidden State (h_t):

The gated output string sent to the autoencoder decoder pass.;

Gating Equations:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

forget

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

input

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

candidate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

output

State Updates:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

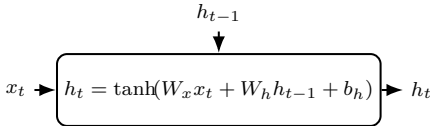
$$h_t = o_t \odot \tanh(c_t)$$

- σ : Sigmoid activation mapping $[0, 1]$
- $\otimes$ / $\odot$: Element-wise matrix multiplication



Beyond abstract optimization behavior, recurrent dynamics can be intuitively analyzed through signal processing as adaptive, multi-band recursive filters.

Vanilla RNN: Scale Interference



- ▶ Multi-scale signals (e.g., $\sin(x) + \sin(nx)$ for large n) are forced to compete for a single hidden state vector h_t .
- ▶ Fast, high-frequency phase shifts continuously scramble and overwrite the slowly evolving macro context.
- ▶ Constant non-linear compression blends overlapping frequencies into a single chaotic trajectory.

LSTM: Spectral Separation The architecture natively partitions features, functioning as a dual-band recursive signal analyzer:

- ▶ **Cell State (c_t) → Low-Pass Integrator**
Purely additive update tracking ($c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$) shields this carousel from fast jitter, cleanly accumulating global long-wave trends.
- ▶ **Hidden State (h_t) → High-Pass Selector**
Gated output routing ($o_t \odot \tanh(c_t)$) captures immediate, sharp transient spikes without corrupting the historical memory baseline.

Application to Denoising: This multi-band separation allows a network to maintain a highly stable global context of ambient light transport across time while staying responsive to instantaneous local disocclusions.



A **ConvLSTM** replaces linear matrix multiplications with 2-D spatial convolutions to preserve the spatial topology of the features across frame transitions.

At the deepest U-Net bottleneck layer, the features are compressed to a low-resolution latent tensor of size $8 \times 8 \times 101$. The gates use a sliding $3 \times 3 \times 101$ volumetric kernel.

ConvLSTM Internal Gate Transforms

$$\mathbf{f}_t = \sigma(W_f * [\mathbf{x}_t, \mathbf{h}_{t-1}] + b_f) \quad (\text{Forget gate drops history at disocclusion})$$

$$\mathbf{i}_t = \sigma(W_i * [\mathbf{x}_t, \mathbf{h}_{t-1}] + b_i) \quad (\text{Input gate drives adaptive updates})$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(W_g * [\mathbf{x}_t, \mathbf{h}_{t-1}] + b_g) \quad (\text{Cell state tracking update})$$

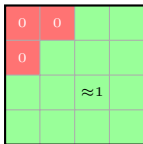
- **Local Gate Activation:** Gates evaluate localized 3×3 neighborhoods independently, allowing the network to drop history ($\mathbf{f}_t \rightarrow 0$) in moving quadrants while accumulating samples in static areas.
- **Receptive Field:** A 3×3 block on the 8×8 grid corresponds to a massive structural footprint in the original rendering space (e.g., 48×48 pixels for a 128×128 input target).



A 3×3 ConvLSTM at the 8×8 U-Net bottleneck delivers three structural properties unavailable to a flat LSTM:

A. Spatially Localised Gates

The 3×3 kernel makes gates *coordinate-aware*. A disocclusion drives $f_t \approx 0$ only at affected cells; the rest keep accumulating:



8×8 bottleneck

No ghosting at red; full accumulation elsewhere.

B. Receptive Field Scaling

Each bottleneck cell covers a 64×64 input patch ($512 \div 8 = 64$). A single 3×3 conv therefore evaluates:

$$(3 \times 64)^2 = 192 \times 192 \text{ px per step.}$$

Shadow boundaries, lighting gradients, and specular highlights all fit inside one convolution pass.

C. Preservation of Structural Flow

Without flattening, spatial topology is preserved at maximum compression. The decoder upsamples cleanly, retaining edges a flat LSTM would discard.



Whether we look at the explicit blending equations of SVGF or the hidden states (h_t) of recurrent neural networks, both frameworks share a deep connection to **Importance Resampling**.

The Continuous State Pool

- ▶ In our basic MC lecture, we saw that **Sampling Importance Resampling (SIR)** evaluates a pool of candidates using normalized importance weights.
- ▶ In spatiotemporal reconstruction, the accumulated historical buffers (or the latent cell states c_t) function as a continuous **temporal particle pool** tracking light transport across frames.

Disocclusion as Weight Collapse

When a structural disocclusion occurs:

- ▶ The target distribution π changes instantaneously.
- ▶ The historical sample pool becomes invalid.
- ▶ Hard-coding $\alpha = 0$ (SVGF) or driving the forget gate to zero ($f_t \rightarrow 0$ in ConvLSTM) is mathematically a **pruning step** that chokes stale history and forces the system to sample from the fresh proposal.

Take-away: Spatiotemporal denoising is a streaming, data-driven resampling engine. Time acts



Contributions and legacy:

- ▶ First demonstration that a neural network can learn stable spatiotemporal integration *without explicit alignment constraints*.
- ▶ The recurrent bottleneck design became the template for **OIDN 2.0**'s temporal mode and super-resolution upscalers.
- ▶ Automated ghosting and occlusion suppression implicitly through standard sequence training pipelines.

Remaining limitations:

- ▶ Geometric warp configurations are completely implicit; tracking performance can drift over long paths.
- ▶ High memory consumption due to continuous h_t state lookups across successive frames.



Definition (Kernel Prediction Foundations)

Instead of predicting a color vector directly, a *Kernel Prediction Network* estimates a discrete matrix of scalar reconstruction weights distributed over a localized support window:

$$\{K_p(q)\}_{q \in \mathcal{N}(p)} = \mathcal{F}_\Theta(I, G, p).$$

A final **Softmax Activation Layer** ensures strict mathematical normalization:

$$K_p(q) \geq 0 \quad \forall q, \quad \sum_{q \in \mathcal{N}(p)} K_p(q) = 1.$$

The Convex Hull Property: Because weights sum to 1 and are non-negative, the denoised result is strictly bounded within the range of the input samples:

$$\tilde{I}(p) \in \text{conv}\{I(q) \mid q \in \mathcal{N}(p)\}.$$

The output cannot contain hallucinated out-of-bounds intensity anomalies.

KPCN⁷ applied deep learning-driven kernel estimation directly to high-quality offline production pipelines:

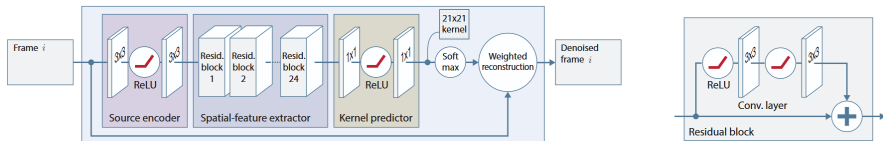


Figure: KPCN channel mapping and modular feature layout extraction loops.

Key design choices:

- **Split diffuse / specular paths:** separate networks handle components independently to isolate discrete noise profiles.
- **Albedo pre-division:** divides input radiance by albedo before denoising to prevent texture detail smearing.
- **21×21 output kernel:** large spatial support predicted per pixel via deep residual pipelines.

⁷S. Bako et al. "Kernel-Predicting Convolutional Networks for Denoising Monte Carlo Renderings". In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 36.4 (2017), 97:1–97:14.

The kernel architecture scales naturally to multi-frame animation sequences by predicting spatiotemporal weights across a sliding window of frames⁸:

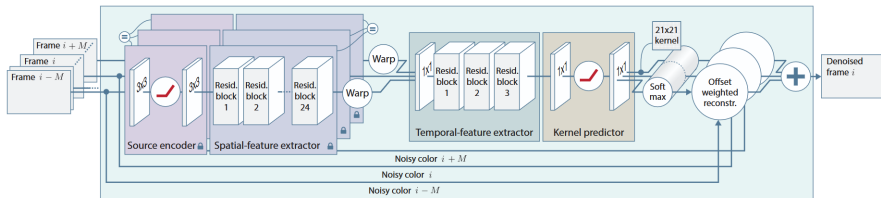


Figure: Symmetric frame windows processing future and past context states.

Definition (Spatiotemporal Kernel Prediction)

Let $K_p^{(j)}(q)$ be the kernel weight predicted for neighbor q in frame $j \in \mathcal{T}$. The denoised pixel is evaluated as:

$$\tilde{I}(p) = \sum_{j \in \mathcal{T}} \sum_{q \in \mathcal{N}(p)} K_p^{(j)}(q) \cdot I_j(q),$$

with a joint softmax normalization constraint over space and time.

⁸T. Vogels et al. "Denoising with Kernel Prediction and Asymmetric Loss Functions". In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 37.4 (2018), 124:1–124:15.

Standard KPN gathers features from pixel-grid averages, which can cause high-frequency detail loss before the filter is applied.

Gathering (Pixel Level):

$$\tilde{I}(p) = \sum_q K_p(q) \bar{I}(q),$$

Applies weights to pre-averaged per-pixel values $\bar{I}(q)$. Information is discarded by early pixel binning.

Splatting (Sample Level): Each raw sample s at coordinate x_s contributes directly to surrounding pixels:

$$\tilde{I}(p) = \sum_{s: p \in \mathcal{N}(x_s)} K_s(p) I_s,$$

Advantages of Sample Splatting:

- ▶ Bypasses lossy pixel-binning stages, preserving raw stochastic sample metrics.
- ▶ Adapts natively to complex distributions (e.g., jittered sampling).
- ▶ Resolves sub-pixel variations, such as crisp highlights and textures.

⁹M. Gharbi et al. "Sample-based Monte Carlo denoising using a kernel-splatting network". In: *ACM Transactions on Graphics (ToG)* 38.4 (2019), pp. 1–12.



The Pixel-Binning Bottleneck: Traditional networks (KPCN/SVGF) collapse stochastic Monte Carlo rays into a pixel average $\bar{I}(q)$ *before* the network runs. This lossy step destroys sub-pixel offsets, geometric silhouettes, and individual path variance.

Pixel Gathering (Strategy 2)

$$\tilde{I}(p) = \sum_{q \in \mathcal{N}(p)} K_p(q) \bar{I}(q)$$

- ▶ Filter is anchored to the discrete **destination pixel center** p .
- ▶ Gathers from neighborhood of pre-blurred, pre-binned pixel averages.

Sample Splatting (Strategy 3)

$$\tilde{I}(p) = \frac{\sum_{s: p \in \mathcal{N}(x_s)} K_s(p) I_s}{\sum_{s: p \in \mathcal{N}(x_s)} K_s(p)}$$

- ▶ Filter is anchored to the continuous **source sample coordinate** x_s Gharbi et al., 2019.
- ▶ Distributes (*splats*) the raw energy I_s of individual rays directly onto pixel grids Gharbi et al., 2019.

Core Edge: Evaluating paths *before* binning preserves sub-pixel variations, enabling the reconstruction of crisp highlights and ultra-fine geometry profiles Gharbi et al., 2019.

Case Study: The Outlier Challenge

To contrast gathering and splatting pipelines, we consider a diagnostic toy task: learning to emulate a target Gaussian blur filter under extreme, non-Gaussian noise conditions¹⁰.

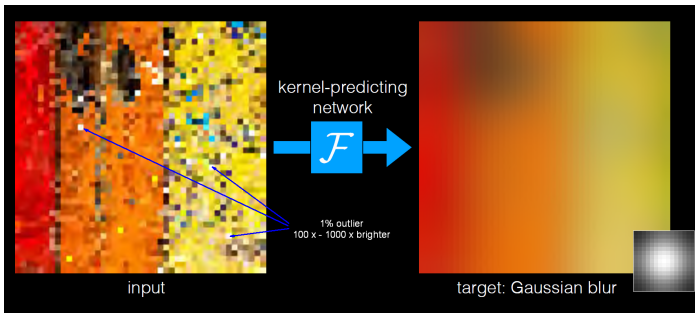


Figure: Input corrupted with 1% extreme specular anomalies.

- ▶ Given clean, uncorrupted input images, a standard image-space network ($\mathcal{F}$) easily learns symmetric, stable blur weights.
- ▶ Introduce a harsh noise profile: **1% of input pixels are corrupted as outliers**, making them 100× to 1000× brighter than their neighbors.
- ▶ This mimics the exact mathematical profile of Monte Carlo ray-tracing fireflies.

¹⁰M. Gharbi et al. "Sample-based Monte Carlo denoising using a kernel-splatting network". In: *ACM Transactions on Graphics (ToG)* 38.4 (2019), pp. 1–12.

Raw stochastic ray samples form an unstructured point cloud with no natural order. Standard CNNs require a rigid, sorted tensor structure. Gharbi et al. (2019) solve this via a symmetric multi-scale loop:

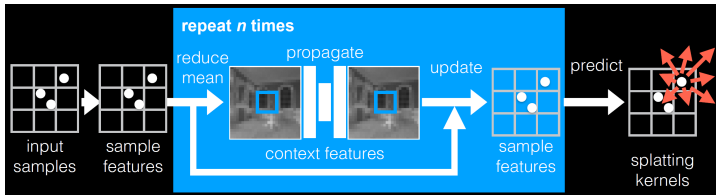


Figure: The end-to-end permutation-invariant sample processing loop Gharbi et al., 2019.

1. **Reduce (Mean):** Sample features undergo a symmetric average operation. Because addition is commutative, the output tensor is completely invariant to the generation order of the rays.
2. **Propagate:** A spatial U-Net processes this order-independent image tensor to extract global scene context, mapping edge fields and lighting gradients.
3. **Update & Predict:** Global context is broadcast back to individual samples via skip connections. Enriched with global awareness, the network predicts custom, asymmetric splatting kernels per ray.



Splatting vs. Gathering: Kernel Footprint Analysis



When a high-energy outlier shifts to an off-center sub-pixel coordinate, the geometric anchoring of the filter determines the stability of the entire reconstruction pass..

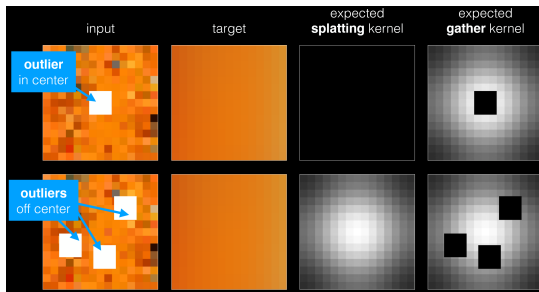


Figure: Grid matrix analysis for off-center outliers: Input, Target, Expected footprints, and Learned network behaviors.

Gathering (Destination-Anchored): Because filters are tied to the pixel center, neighboring target cells must deform their shapes into complex, asymmetric shields to block the nearby firefly. Tracking imperfections leak energy, causing blotches.

Splatting (Source-Anchored): The clean center sample keeps a perfect, un-deformed Gaussian kernel. The moving outlier sample carries its own learned zero-weight dampener directly on its coordinate, completely shielding the rest of the image space.



Evaluating filters at the sample level changes how networks defend against high-energy Monte Carlo anomalies (*fireflies*).

Gather Networks: Spatial Shielding

- ▶ When a destination pixel sits near a $1000\times$ bright firefly, its gather kernel must manually deform to carve a shield around it.
- ▶ **Failure Mode:** Every neighboring pixel must coordinate complex, asymmetric masks perfectly. Any tracking drift leaks massive outlier energy, creating a **blotchy, unstable output**.

Splat Networks: Source Dampening

- ▶ The network evaluates the anomalous sample directly, noting its variance against the shared context features.
- ▶ **Surgical Neutralization:** The model forces the outlier's unique splatting kernel to a **zero-weight footprint** ($K_s \rightarrow 0$). The firefly is choked at the source and never broadcasts into the image.

Pedagogical Summary: Gather denoisers fight noise by building unstable defensive walls from the outside; splatting denoisers turn off the noise directly at the source ray.



The theoretical and practical unification of recurrent loops and kernel prediction was formalized by NVIDIA Research in 2020.

Milestone Paper

Hasselgren et al. (2020): “Neural Temporal Adaptive Sampling and Denoising”
Published at *ACM Transactions on Graphics (Proc. SIGGRAPH 2020)*.

Core Contributions to Modern Reconstruction:

- ▶ **Causal Recurrent Architecture:** Replaced non-causal multi-frame sliding windows with a strict real-time friendly, forward-facing recurrent feedback loop.
- ▶ **Hierarchical Kernel Prediction Heads:** Instead of predicting pixel colors directly, the decoder ends in multiple multi-scale prediction branches.
- ▶ **Dual Spatiotemporal Filtering:** At the final reconstruction pass, the network outputs a spatial kernel to smooth current frame noise and a temporal kernel to intelligently blend recurrent history.



Intel Open Image Denoise (OIDN): Industry Standard



OIDN¹¹ is an open-source (Apache 2.0) production-grade reconstruction framework integrated across major industry renderers (Blender Cycles, Autodesk Arnold, LuxCore).



Figure: Production render comparison: 4 spp noisy stochastic viewport input (left) vs. completed deep spatiotemporal reconstruction pass (right).

Core Architecture Features:

- ▶ **Multi-Channel Processing:** Independent deep sub-networks optimize feature weights separately for beauty paths, albedo targets, and surface normal planes.
- ▶ **Linear HDR Space Training:** Filter optimizations occur entirely in uncompressed linear light space to prevent exposure clipping.

Deployment & Performance:

- ▶ **Hardware Portability:** High cross-platform performance over heterogeneous CPU and GPU architectures via native CUDA, Metal, and SYCL backends.
- ▶ **Learned Temporal Mode:** Replaces older, hand-crafted accumulation constraints with data-driven spatiotemporal sequencing.

¹¹A. T. Áfrá et al. *Intel Open Image Denoise*. <https://www.openimagedenoise.org>. Open-source deep learning denoiser, Apache 2.0 licence. 2022.



Modern versions of OIDN move beyond single-frame autoencoders to realize the full academic hybrid synthesis (Causal Recurrent Loop + Local Spatial Kernel Prediction).

Step 1: Path Splitting & Demodulation

The noisy beauty radiance buffer is separated into independent **Direct** (shadow penumbras) and **Indirect** (global illumination bounce) channels. Each path is **demodulated** by its G-buffer albedo to cleanly isolate the raw light field: $L = C/\rho$.

Step 2: Recurrent Spatiotemporal U-Net

A deep convolutional network augmented with a forward-facing temporal feedback loop processes the flat light fields and variance maps. Instead of directly synthesizing pixel colors, it predicts normalized local diffuse/specular filters.

Step 3: Texture Re-modulation

The filtered, noise-free illumination components ($\tilde{L}$) are recombined and multiplied back by the sharp, noise-free G-buffer texture maps to reconstruct the final pixel color:
 $\tilde{C} = \rho \cdot \tilde{L}$.



The **OptiX AI Denoiser** is a production framework shipped with NVIDIA's OptiX ray-tracing SDK and exposed through the OptiX 7 API. While its internal architecture remains proprietary, its functional design aligns with the deep temporal autoencoder paradigms pioneered by Chaitanya et al. (2017).

Publicly Disclosed Behavior:

- ▶ Utilizes deep neural networks accelerated directly by specialized **Tensor Cores** via optimized inference engines.
- ▶ Replaces traditional, hand-crafted temporal α -blending with data-driven spatiotemporal path integration.

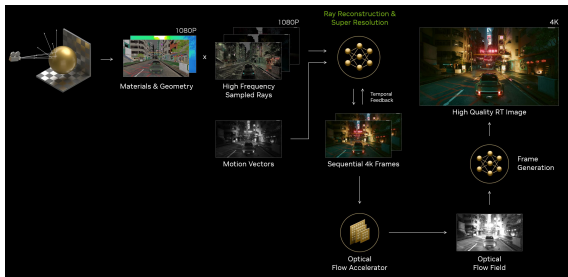
Ingested API Buffers:

- ▶ Noisy beauty radiance targets
- ▶ Diffuse Albedo G-Buffer
- ▶ Surface Normal Plane vectors
- ▶ Color history frames and screen-space motion fields

Academic Context: OptiX serves as a prime real-time industry instantiation of utilizing structural geometric buffers to guide deep spatial filter weights.



Case Study: NVIDIA DLSS 3.5 Ray Reconstruction



NVIDIA's public specification states that **Ray Reconstruction (DLSS 3.5)** replaces multiple hand-tuned, cascaded denoisers with a single unified AI model to preserve ray-traced effects before upscaling.

The Multitask AI Paradigm:

- **Consolidation:** Eradicates the traditional pipeline conflict where an independent spatial denoiser destroys high-frequency details before the upscaler runs.
- **Unified Inference:** Denoising, time tracking, and super-resolution are executed inside a single model.

Conceptual Lineage:

- Represents the industrial evolution of recurrent configurations Chaitanya et al., 2017 and spatial kernel prediction Vogels et al., 2018.
- **Sample-Space Alignment:** Conceptually mirrors sample-based research (like Gharbi et al. (2019)) by utilizing raw ray data to resolve sub-pixel variations.



The modern state-of-the-art in reconstruction resolves the limitations of standalone networks by converging on a unified, co-trained architectural paradigm.

The Grand Unification: Recurrent Spatiotemporal Reconstruction

Modern frameworks unify continuous **Temporal Memory** (to handle frame-to-frame variance) with data-driven **Spatial Filters** (to enforce localized structural sharpness).

1. The Temporal Core

- ▶ Powered by causal recurrent feedback loops (**ConvLSTM** / **GRU** structures).
- ▶ Tracks macro-scale light transport and radiance moments over time.
- ▶ **Impact:** Eliminates high-frequency "boiling" artifacts and ensures absolute temporal stability.

2. The Spatial Core (Two Paradigms)

Instead of direct synthesis, the temporal state feeds a decoder that branches into localized filters:

- ▶ **Pixel-Gathering Kernels:**
Anchored to discrete *destination pixel centers* to blend neighborhood pixel-bin averages (e.g., **Hasselgren et al., Intel OIDN 2.x**).
- ▶ **Sample-Space Processing (Conceptual):**
Interrogating raw, un-binned ray elements to bypass pixel-averaging information loss (conceptually mirrored in modern unified pipelines like **NVIDIA DLSS 3.5**).

Take-away: Time handles structural continuity; Space handles geometric sharpness.

Comparison: Reconstructive Filtering Methods

Property	Cross-Bilateral	SVGF ¹²	KPN / KPCN ¹³	Recurrent Autoencoder ¹⁴
Output Strategy	Weighted Avg.	Weighted Avg.	Learned Kernel	Direct Synthesis
Energy Enforced	✓	✓	✓	×
Temporal Mode	Static (None)	Causal EMA Loop	Symmetric Window	Latent ConvLSTM State
Disocclusion Check	None	Geometric Deltas	Context-Driven	Learned Gating Fields
Adaptivity Parameter	Manual Heuristics	Variance-Guided	Data-Driven	Data-Driven
Training Dataset	None Required	None Required	Supervised Pairs	Supervised Pairs
Execution Budget	Low (< 0.5ms)	Medium (≈ 1.5 ms)	High / Offline	Medium (≈ 2.0 ms)

- **Kernel Methods** Vogels et al., 2018: Leverage the convex hull property to prevent color artifacting, making them a popular choice for production pipelines.
- **Recurrent Architectures** Chaitanya et al., 2017: Replace hand-tuned edge-stopping functions with data-driven temporal weighting and disocclusion handling.
- **Modern Real-Time Pipelines:** Integrate ray reconstruction Hasselgren et al., 2020 and super-resolution into unified networks to maximize performance and image sharpness.

¹²C. Schied et al. "Spatiotemporal Variance-Guided Filtering: Real-Time Reconstruction for Path-Traced Global Illumination". In: *Proceedings of High Performance Graphics (HPG)*. ACM, 2017, pp. 1–12.

¹³S. Bako et al. "Kernel-Predicting Convolutional Networks for Denoising Monte Carlo Renderings". In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 36.4 (2017), 97:1–97:14.

¹⁴C. R. A. Chaitanya et al. "Interactive Reconstruction of Monte Carlo Image Sequences using a Recurrent Denoising Autoencoder". In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 36.4 (2017), 98:1–98:12.



Modern filtering has shifted from hand-designed heuristics to data-driven physical reconstruction.

- ▶ **G-Buffers are the foundation** Nehab et al., 2007: Geometric and material priors allow us to distinguish real edges from random Monte Carlo variance.
- ▶ **Demodulation is mandatory**: Decoupling illumination from albedo is the only way to preserve high-frequency texture details while clearing heavy noise.
- ▶ **Time is the ultimate constraint**: Stable denoising at 1 spp requires a recurrent "memory" of the scene's light transport.
- ▶ **Hybrid is the standard** Hasselgren et al., 2020: The most robust systems today use recurrent states for time and kernel prediction for space.

Industry Shift: We no longer simply "blur" noise; we stochastically **reconstruct** light transport from sparse data.



- [1] A. T. Áfrá et al. *Intel Open Image Denoise*.
<https://www.openimagedenoise.org>. Open-source deep learning denoiser, Apache 2.0 licence. 2022.
- [2] S. Bako, T. Vogels, B. McWilliams, M. Meyer, J. Novák, A. Harvill, P. Sen, T. Deroose, and F. Rousselle. “Kernel-Predicting Convolutional Networks for Denoising Monte Carlo Renderings”. In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 36.4 (2017), 97:1–97:14.
- [3] C. R. A. Chaitanya, A. S. Kaplanyan, C. Schied, M. Salvi, A. Lefohn, D. Nowrouzezahrai, and T. Aila. “Interactive Reconstruction of Monte Carlo Image Sequences using a Recurrent Denoising Autoencoder”. In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 36.4 (2017), 98:1–98:12.
- [4] M. Gharbi, T.-M. Li, M. Aittala, J. Lehtinen, and F. Durand. “Sample-based Monte Carlo denoising using a kernel-splatting network”. In: *ACM Transactions on Graphics (ToG)* 38.4 (2019), pp. 1–12.
- [5] J. Hasselgren, J. Munkberg, M. Salvi, A. Lefohn, and T. Akenine-Möller. “Neural Temporal Adaptive Sampling and Denoising”. In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 39.4 (2020), 147:1–147:14.



- [6] D. Nehab, P. V. Sander, J. Lawrence, N. Tatarchuk, and J. R. Isidoro. “Accelerating Real-Time Shading with Reverse Reprojection Caching”. In: *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Graphics Hardware*. 2007, pp. 25–35.
- [7] O. Ronneberger, P. Fischer, and T. Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, 2015, pp. 234–241.
- [8] C. Schied, A. Kaplanyan, C. Wyman, A. Patney, C. R. A. Chaitanya, J. Burgess, S. Liu, C. Dachsbacher, A. Lefohn, and M. Salvi. “Spatiotemporal Variance-Guided Filtering: Real-Time Reconstruction for Path-Traced Global Illumination”. In: *Proceedings of High Performance Graphics (HPG)*. ACM, 2017, pp. 1–12.
- [9] C. E. Shannon. “Communication in the presence of noise”. In: *Proceedings of the IRE* 37.1 (1949), pp. 10–21.
- [10] C. Tomasi and R. Manduchi. “Bilateral Filtering for Gray and Color Images”. In: *Proceedings of the Sixth International Conference on Computer Vision (ICCV)*. IEEE, 1998, pp. 839–846.



- [11] T. Vogels, F. Rousselle, B. McWilliams, G. R  thlin, A. Harvill, D. Adler, M. Meyer, and J. Nov  k. “Denoising with Kernel Prediction and Asymmetric Loss Functions”. In: *ACM Transactions on Graphics (Proc. SIGGRAPH)* 37.4 (2018), 124:1–124:15.